

JavaScript (basics)

This slide is copied and modified from
Fulvio Corno, Luigi De Russis



Politecnico
di Torino



Goal

- Learn JavaScript as a language
- Understand the specific semantics and programming patterns
 - We assume a programming knowledge in other languages
- Updated to ES6 (2015) language features
- Supported by server-side (Node.js) and client-side (browsers) run-time environments
 - More recent language additions also supported (through **transpiling**)



Outline

- What is JavaScript?
- History and versions
- Language structure
- Types, variables
- Expressions
- Control structures
- Arrays
- Strings



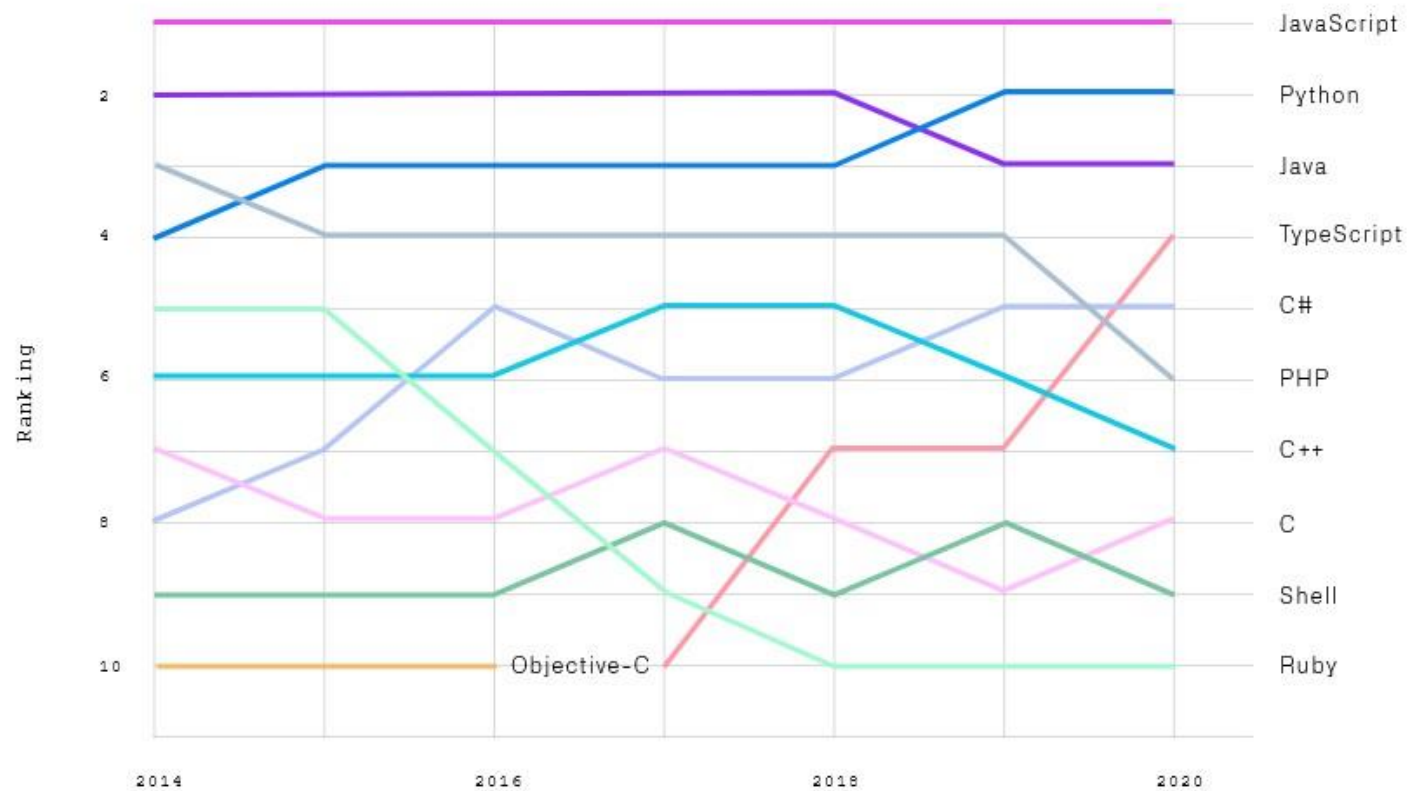
- Function
 - Methods
- Value and type
- variables

WHAT IS JAVASCRIPT?

JavaScript – The language of the Web

// The languages that dominated

Top languages over the years





JavaScript

- JavaScript (JS) is a programming language
- It is currently the only programming language that a browser can execute natively...
- ... and it also run on a computer, like other programming languages (thanks to Node.js)
- It has nothing to do with Java
 - Named that way for marketing reasons, only
- The first version was written in 10 days (!)
 - Several fundamental language decisions were made because of company politics and not technical reasons!

HISTORY AND VERSIONS

JavaScript – The language of the Web

JAVASCRIPT VERSIONS



Brendan Eich

- ▶ **JAVASCRIPT (December 4th 1995)** Netscape and Sun press release
- ▶ **ECMAScript Standard Editions:** <https://www.ecma-international.org/ecma-262/> 
- ▶ **ES1 (June 1997)** Object-based, Scripting, Relaxed syntax, Prototypes
- ▶ **ES2 (June 1998)** Editorial changes for ISO 16262
- ▶ **ES3 (December 1999)** Regexp, Try/Catch, Do-While, String methods
- ▶ **ES5 (December 2009)** Strict mode, JSON, .bind, Object mts, Array mts
- ▶ **ES5.1 (June 2011)** Editorial changes for ISO 16262:2011
- ▶ **ES6 (June 2015)** Classes, Modules, Arrow Fs, Generators, Const/Let, Destructuring, Template Literals, Promise, Proxy, Symbol, Reflect
- ▶ **ES7 (June 2016)** Exponentiation operator (**) and Array Includes
- ▶ **ES8 (June 2017)** Async Fs, Shared Memory & Atomics

10
yrs

Main
target

ES9,
ES10,
...

Also: ES2015

Also: ES2016

Also: ES2017



JavaScript versions

- ECMAScript (also called ES) is the official name of JavaScript (JS) standard
- ES6, ES2015, ES2016 etc. are implementations of the standard
- All browsers used to run ECMAScript 3
- ES5, and ES2015 (=ES6) were huge versions of JavaScript
- Then, yearly release cycles started
 - By the committee behind JS: TC39, backed by Mozilla, Google, Facebook, Apple, Microsoft, Intel, PayPal, Salesforce, etc.
- ES2015 (=ES6) is covered in this course



Official ECMA standard (formal and unreadable)

Search...

TABLE OF CONTENTS

- Introduction
- 1 Scope
- 2 Conformance
- 3 Normative References
- 4 Overview
- 5 Notational Conventions
- 6 ECMAScript Data Types and Values
- 7 Abstract Operations
- 8 Executable Code and Execution Contexts
- 9 Ordinary and Exotic Objects Behaviours
- 10 ECMAScript Language: Source Code
- 11 ECMAScript Language: Lexical Grammar
- 12 ECMAScript Language: Expressions
- 13 ECMAScript Language: Statements and Declarations
- 14 ECMAScript Language: Functions and Classes
- 15 ECMAScript Language: Scripts and Modules
- 16 Error Handling and Language Extensions
- 17 ECMAScript Standard Built-in Objects
- 18 The Global Object
- 19 Fundamental Objects
- 20 Numbers and Dates
- 21 Text Processing
- 22 Indexed Collections
- 23 Keyed Collections
- 24 Structured Data
- 25 Control Abstraction Objects
- 26 Reflection
- 27 Memory Model
- A Grammar Summary
- B Additional ECMAScript Features for Web Browsers
- C The Strict Mode of ECMAScript
- D Corrections and Clarifications in ECMAScript 2015 wit...
- E Additions and Changes That Introduce Incompatibiliti...
- F Colophon
- G Bibliography
- H Copyright & Software License

ecma
INTERNATIONAL

ECMA-262, 10th edition, June 2019
ECMAScript® 2019 Language Specification

Contributing to this Specification

This specification is developed on GitHub with the help of the ECMAScript community. There are a number of ways to contribute to the development of this specification:

GitHub Repository: <https://github.com/tc39/ecma262>
Issues: All Issues, File a New Issue
Pull Requests: All Pull Requests, Create a New Pull Request
Test Suite: Test262

Editors:

- Brian Terlson (@bterlson)
- Bradley Farias (@bradleybeck)
- Jordan Harband (@ljharb)

Community:

- Mailing list: es-discuss
- IRC: #tc39 on freenode

Refer to the colophon for more information on how this document is created.

Introduction



JavaScript Engines

Browser

Javascript Engine

Chrome



V8 Engine
(OpenSource)

Firefox



Spider Monkey
(OpenSource)

Safari



JavaScriptCore
(OpenSource)

Internet
Explorer



Chakra
(Proprietary)



JS Compatibility

- JS is backwards-compatible
 - Once something is accepted as valid JS, there will not be a future change to the language that causes that code to become invalid JS
 - TC39 members: "we don't break the web!"
- JS is not forwards-compatible
 - New additions to the language will not run in an older JS engine and may crash the program

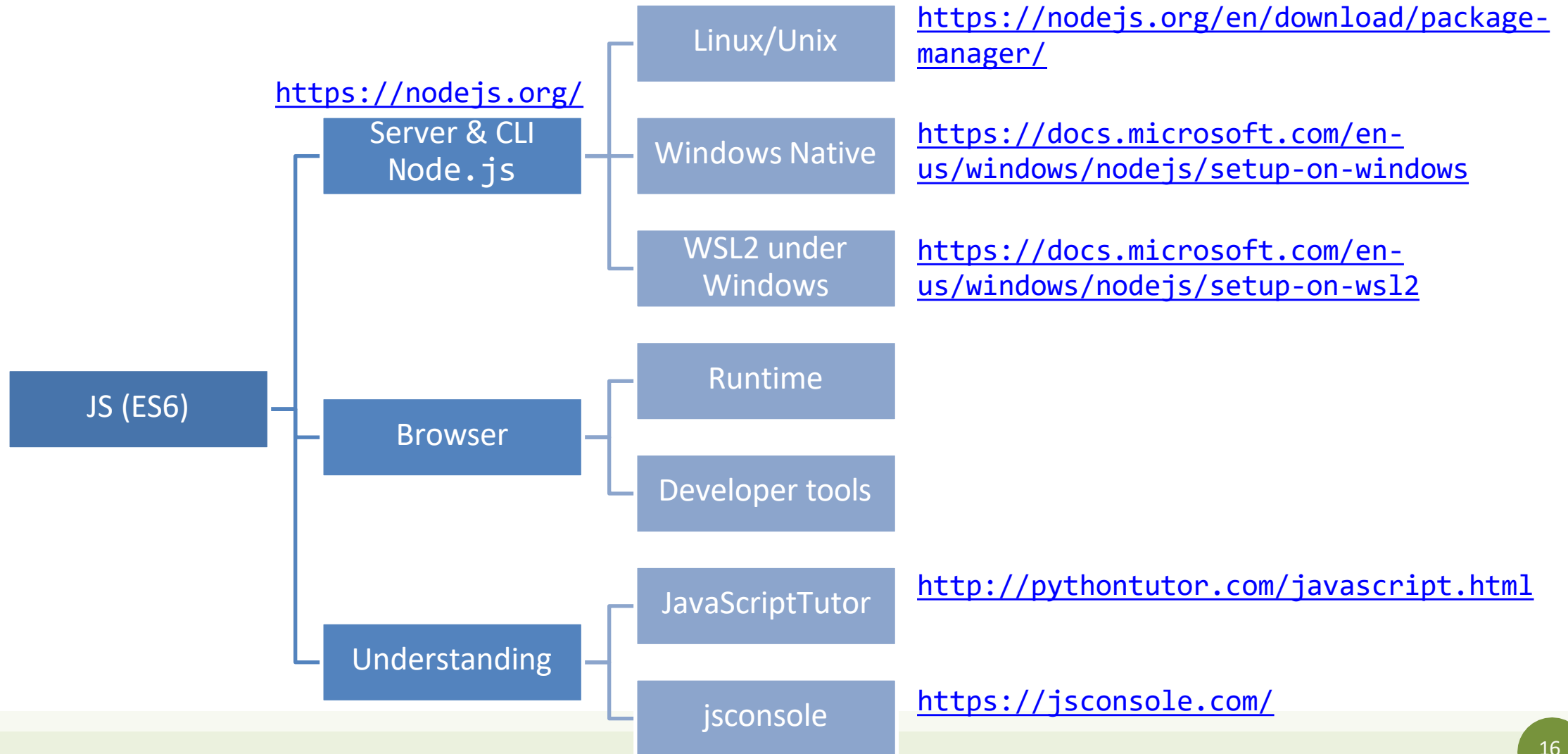


JS Compatibility

- Strict mode was introduced to disable very old (and dangerous) semantics
- Supporting multiple versions is achieved by:
 - Transpiling – Babel (<https://babeljs.io>) converts from newer JS syntax to an equivalent older syntax
 - Polyfilling – user- (or library-) defined functions and methods that “fill” the lack of a feature by implementing the newest available one



JS Execution Environments





JavaScriptTutor

Write code in JavaScript ES6 (drag lower right corner to resize code editor)

```
1 let nome = "Fulvio" ;
2 let cognome = "Corno" ;
3
4 function hello(c, n) {
5   n = n || "sig."
6   const saluto = n + " " + c ;
7   return saluto ;
8 }
9
10 let s1 = hello(cognome, nome)
11 let s2 = hello(nome)
12
13 let nome2 = [...nome]
14 let cognome2 = [...cognome]
```

→ line that just executed

→ next line to execute

<< First < Prev Next > Last >>

Done running (16 steps)

Frames

Global frame	
hello	
nome	"Fulvio"
cognome	"Corno"
s1	"Fulvio Corno"
s2	"sig. Fulvio"
nome2	
cognome2	

Objects

```
function hello(c, n) {
  n = n || "sig."
  const saluto = n + " " + c ;
  return saluto ;
}
```

array

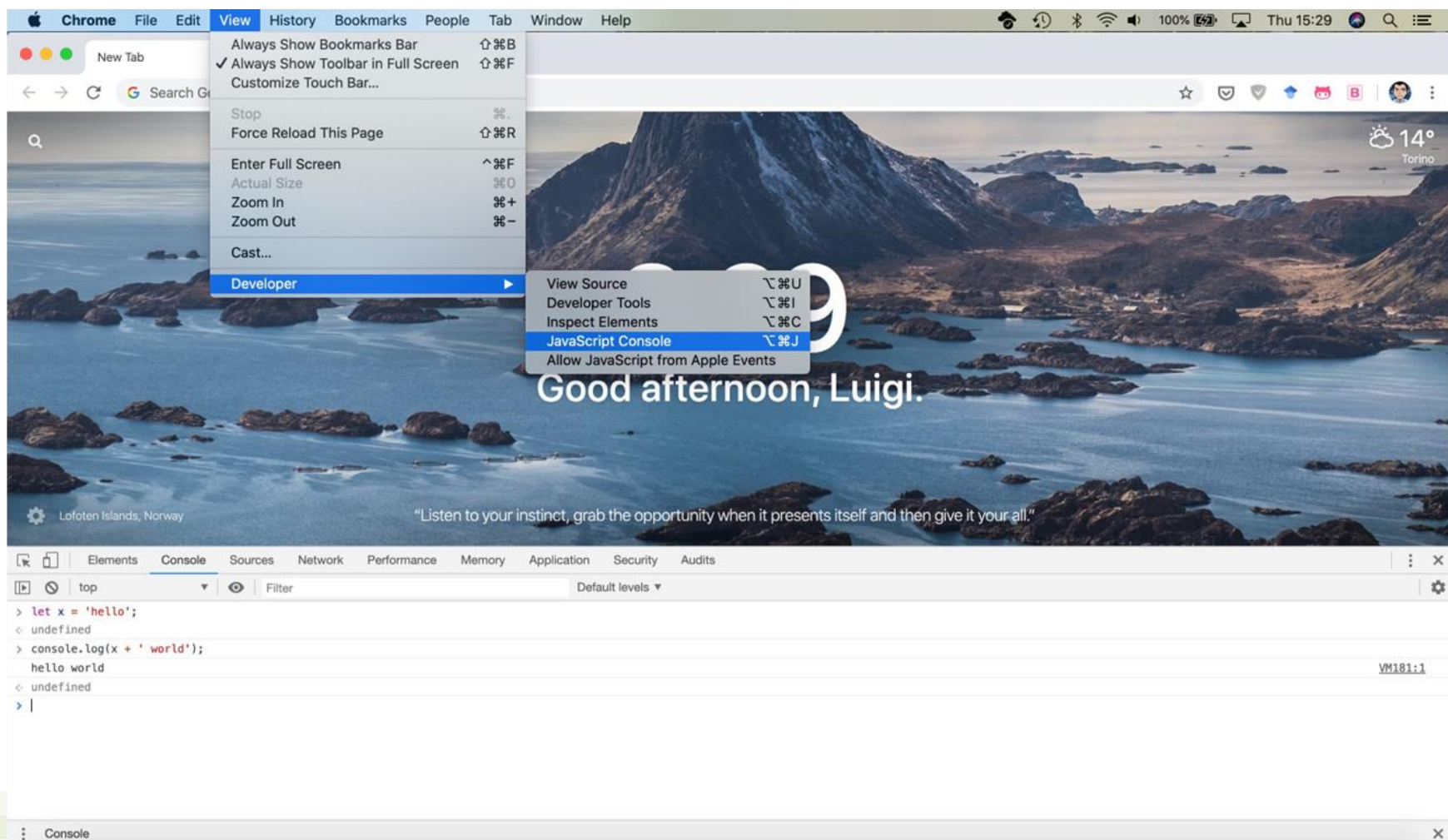
0	1	2	3	4	5
"F"	"u"	"l"	"v"	"i"	"o"

array

0	1	2	3	4
"C"	"o"	"r"	"n"	"o"



Browser and JS console



LANGUAGE (Lexical) STRUCTURE

The set of elementary rules that specifies how you write programs in that language, meaning that the lowest-level syntax of a language



Coverage

- Case sensitivity, spaces, and line breaks
- Comments
- Literals
- Identifiers and reserved words
- Unicode
- Optional semicolons





Lexical Structure

- One File = One JS program
 - Each file is loaded independently and
 - Different files/programs may communicate through global state
 - The “module” mechanism extends that (provides state sharing in a clean way)
- The file is entirely parsed, and then executed from top to bottom
- Relies on a standard library
 - and many additional APIs provided by the execution environment



Lexical Structure

```
> let x = '😬';  
< undefined  
> console.log(x);  
😬
```

```
> let ööö = 'appalled'  
> ööö  
'appalled'
```

- JavaScript is written in Unicode (do not abuse), so it also supports non-latin characters for names and strings, even emoji
- Semicolons (;) are not mandatory (automatically inserted)
- Case sensitive
- Comments as in C (`/*..*/` and `//`)
- Literals and identifiers (start with letter, \$, _)
 - A literal is a data value that appears directly in a program
- Some reserved words
- C-like syntax



Reserved

break, case, catch, continue, default, delete, do, else, false, finally, for, function, if, in, instanceof, new, null, return, switch, this, throw, true, try, typeof, var, void, while, with, abstract, boolean, byte, char, class, const, debugger, double, enum, export, extends, final, float, goto, implements, import, int, interface, long, native, package, private, protected, public, short, static, super, synchronized, throws, transient, volatile, as, is, namespace, use, arguments, Array, Boolean, Date, decodeURI, decodeURIComponent, encodeURI, Error, escape, eval, EvalError, Function, Infinity, isFinite, isNaN, Math, NaN, Number, Object, parseFloat, parseInt, RangeError, ReferenceError, RegExp, String, SyntaxError, TypeError, undefined, unescape, URIError ...



Semicolon (;)

```
1. let a  
2. a  
3. =  
4. 3  
5. console.log(a)
```

```
1. let a; a = 3; console.log(a);
```

- Can omit the semicolon if two statements are written on separate lines.
- JS inserts them as needed
 - When next line starts with code that breaks the current one
 - When the next line starts with }
 - When there is return, break, throw, continue on its own line
- Be careful that forgetting semicolon can lead to unexpected behavior
 - A newline does not automatically insert a semicolon: if the next line starts with (or [, it is interpreted as function call or array access
- We will loosely follow the Google style guide, so we will always insert semicolons after each statement
 - <https://google.github.io/styleguide/jsguide.html>



Strict Mode

```
// first line of file  
"use strict" ;  
// always!!
```

- Directive introduced in ES5: **"use strict"** ;
 - Compatible with older version (it is just a string)
- Code is executed in strict mode
 - This fixes some important language deficiencies and provides stronger error checking and security
- Examples:
 - Fixes mistakes that make it difficult for JavaScript engines to perform optimizations
 - Strict mode code can sometimes be made to run faster than code without
 - Eliminates some JavaScript silent errors by changing them to throw errors
 - Functions invoked as functions and not as methods of an object have this undefined
 - Cannot define 2 or more properties or function parameters with the same name



TYPES AND VARIABLES

values are represented and manipulated in a programming language are known as types.

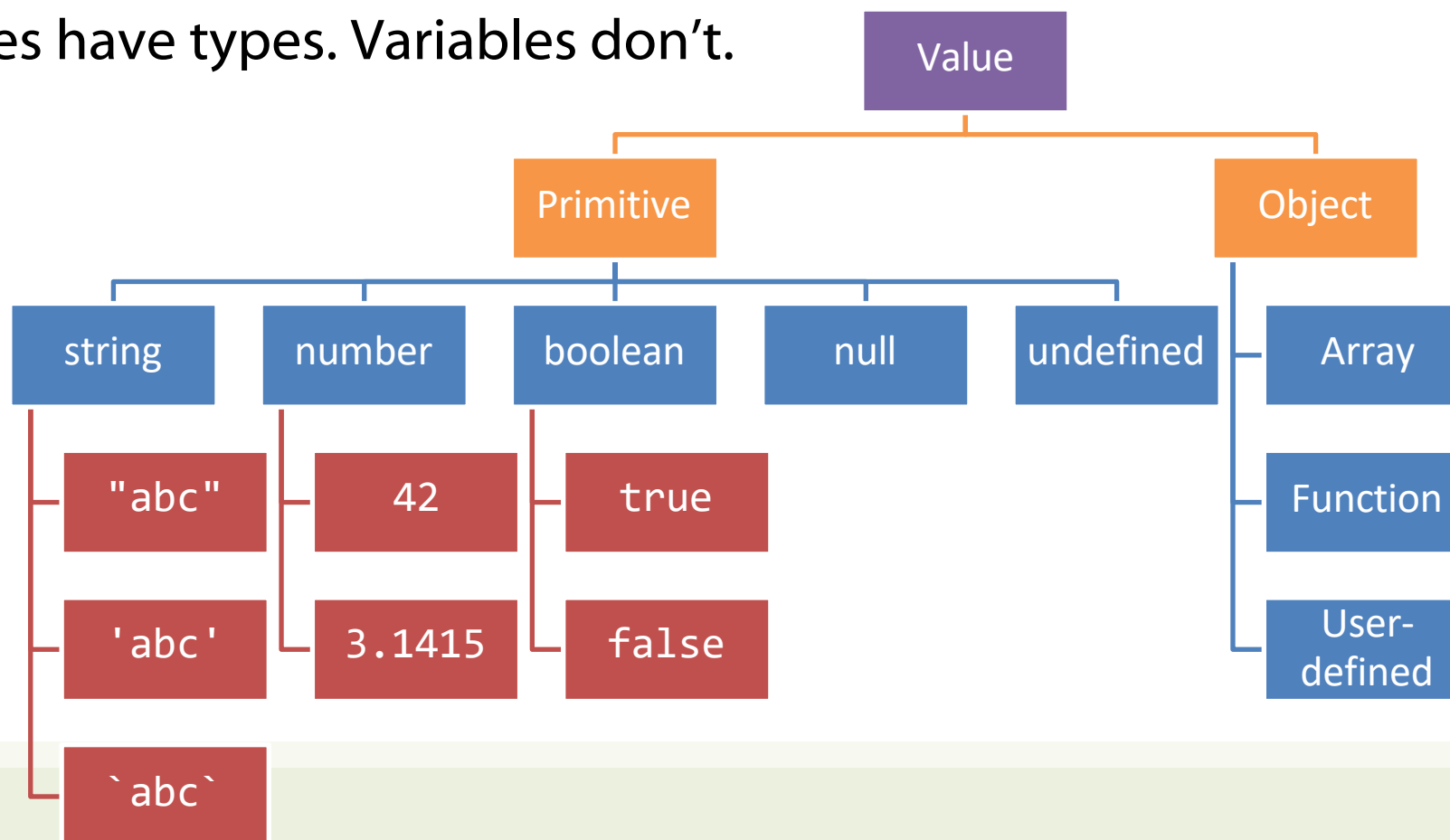
When a program needs to retain a value for future use, it assigns the value to a variable



Values and Types

*Values have types.
Variables don't.*

- ES6 adds a new special-purpose type, known as Symbol
- Values have types. Variables don't.





Primitives and Objects

- Primitives are immutable
 - They do not have methods or properties that can alter them
 - Strings can be thought of as arrays of characters, and you might expect them to be mutable
 - Can access the text at any index of a string, but JavaScript provides no way to alter the text of an existing string
- All primitives in any programming language are passed by value
- Objects are all other JavaScript elements
 - Literals, arrays, dates, etc.
 - They are mutable, meaning that certain methods can alter them
- All non-primitives are passed by reference
- Collection of properties where each property has a name and a value
 - Either a primitive value or another object
- Set, Map



Boolean, true-truthy, false-falsy, comparisons

- 'boolean' type with literal values: **true**, **false**
- When converting to Boolean,
 - Following values are '**falsy**'
 - 0, -0, NaN, undefined, null, '' (empty string)
 - Every other value is '**truthy**'
 - 3, 'false', [] (empty array), {} (empty object)
- Booleans and comparisons
 - **a == b** convert types and compare results
 - **a === b** inhibit automatic type conversion and compare results

```
> Boolean(3)
true
> Boolean('')
false
> Boolean(' ')
true
```



Number

- Can represent -2^{53} and $+2^{53}$
- No distinction between integers and reals
- Automatic conversions according to the operation
- JS works with number using arithmetic operations
 - $+$, $-$, $*$, $\%$, $/$
- JS supports more complex operations through Math object
 - `Math.pow(2, 53)`
 - `Math.round(.6)`
- Arbitrary precision integers with "BigInt"
 - BigInt literals are written as a string of digits followed by a lowercase letter n
 - `123456789n`



Special Values

- **undefined**: variable declared but NOT initialized
 - Detect with: `typeof variable === 'undefined'`
 - `void x` always returns `undefined`
- **null**: an empty value, used to indicate the absence of a value
- Null and Undefined are called nullish values
- **NaN (Not a Number)**
 - It is actually a number
 - Invalid output from arithmetic operation or parse operation



Variables

```
> v = 7 ;  
7  
> v = 'hi' ;  
'hi'
```

- Variables are pure references: they refer to a value
- The same variable may refer to different values (even of different types) at different times
- Declaring a variable:
 - `let`
 - `const`
 - `var`

```
> let a = 5  
> const b = 6  
> var c = 7  
> a = 8  
8  
> b = 9  
Thrown:  
TypeError: Assignment to  
constant variable.  
> c = 10  
10
```



Variable declarations

Declarator	Can reassign?	Can re-declare?	Scope	Hoisting *	Note
let	Yes	No	Enclosing block {...}	No	<i>Preferred</i>
const	No §	No	Enclosing block {...}	No	<i>Preferred</i>
var	Yes	Yes	Enclosing function, or global	Yes, to beginning of function or file	<i>Legacy, beware its quirks, try not to use</i>
None (implicit)	Yes	N/A	Global	Yes	<i>Forbidden in strict mode</i>

§ Prevents reassignment (a=2), does not prevent changing the value of the referred object (a.b=2)

* Hoisting = “lifting up” the definition of a variable (not the initialization!) to the top of the current scope (e.g., the file or the function)



Scope

```
1. "use strict" ;  
2. let a = 1;  
3. const b = 2;  
4. let c = true;  
5. let a = 5;  
6. // SyntaxError: Identifier 'a' has already been declared
```



Scope

- Each { } is called a **block**
- 'let' and 'const' variables are block-scoped.
- They exist only in their defined and inner scopes
 - Typically, you don't create a new scope in this way!

```
1. "use strict" ;  
2. let a = 1;  
3. const b = 2;  
4. let c = true;  
5. {  
6.     // creating a new scope...  
7.     let a = 5 ;  
8.     console.log(a) ;  
9. }  
  
10. console.log(a) ;
```



Scope and Hoisting

```
1. "use strict" ;
2. function example(x) {
3.     let a = 1;          // var c; // hoisted
4.     console.log( a ); // 1
5.     console.log( b ); // ReferenceError: b is not defined
6.     console.log( C ); // undefined
7.     if( x > 1) {
8.         let b = a + 1;
9.         var c = a * 2;
10.    console.log( a ): // 1
11.    console.log( b ); // ReferenceError: b is not defined
12.    console.log( c ); 2
13. }
14.example( 2 );
```



JavaScript: The Definitive Guide, 7th Edition
Chapter 2. Types, Values, and Variables
Chapter 3. Expressions and Operators

Mozilla Developer Network
JavaScript Guide » Expressions and operators

JavaScript – The language of the Web

EXPRESSIONS



Operators



Full reference and operator precedence:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Operator_Precedence#Table

- Assignment operators
- Comparison operators
- Arithmetic operators
- Bitwise operators
- Logical operators
- String operators
- Conditional (ternary) operator
- Comma operator
- Unary operators
- Relational operators



Assignment

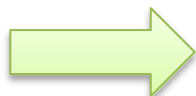
- `let variable = expression ;`
 - `// declaration with initialization`
- `variable = expression ;`
 - `// reassignment`

Name	Shorthand operator	Meaning
Assignment	<code>x = y</code>	<code>x = y</code>
Addition assignment	<code>x += y</code>	<code>x = x + y</code>
Subtraction assignment	<code>x -= y</code>	<code>x = x - y</code>
Multiplication assignment	<code>x *= y</code>	<code>x = x * y</code>
Division assignment	<code>x /= y</code>	<code>x = x / y</code>
Remainder assignment	<code>x %= y</code>	<code>x = x % y</code>
Exponentiation assignment 🔥	<code>x **= y</code>	<code>x = x ** y</code>
Left shift assignment	<code>x <<= y</code>	<code>x = x << y</code>
Right shift assignment	<code>x >>= y</code>	<code>x = x >> y</code>
Unsigned right shift assignment	<code>x >>>= y</code>	<code>x = x >>> y</code>
Bitwise AND assignment	<code>x &= y</code>	<code>x = x & y</code>
Bitwise XOR assignment	<code>x ^= y</code>	<code>x = x ^ y</code>
Bitwise OR assignment	<code>x = y</code>	<code>x = x y</code>



Comparison Operators

Operator	Description	Examples returning true
Equal (==)	Returns <code>true</code> if the operands are equal.	<code>3 == var1</code> <code>"3" == var1</code> <code>3 == '3'</code>
Not equal (!=)	Returns <code>true</code> if the operands are not equal.	<code>var1 != 4</code> <code>var2 != "3"</code>
Strict equal (===)	Returns <code>true</code> if the operands are equal and of the same type. See also <code>Object.is</code> and sameness in JS .	<code>3 === var1</code>
Strict not equal (!==)	Returns <code>true</code> if the operands are of the same type but not equal, or are of different type.	<code>var1 !== "3"</code> <code>3 !== '3'</code>
Greater than (>)	Returns <code>true</code> if the left operand is greater than the right operand.	<code>var2 > var1</code> <code>"12" > 2</code>
Greater than or equal (>=)	Returns <code>true</code> if the left operand is greater than or equal to the right operand.	<code>var2 >= var1</code> <code>var1 >= 3</code>
Less than (<)	Returns <code>true</code> if the left operand is less than the right operand.	<code>var1 < var2</code> <code>"2" < 12</code>
Less than or equal (<=)	Returns <code>true</code> if the left operand is less than or equal to the right operand.	<code>var1 <= var2</code> <code>var2 <= 5</code>





Comparing Objects

- Double equals (==) will perform a type conversion
- Triple equals (===) will do the same comparison as double equals but without type conversion
- Comparison between objects with == or === compares the **references** to object
 - True only if they are the same object
 - False if they are identical objects
- Comparison with < > <= >= first converts object and compares values
 - Into a Number, or more likely a String

```
> a={x:1}  
{ x: 1 }
```

```
> b={x:1}  
{ x: 1 }
```

```
> a===b  
false
```

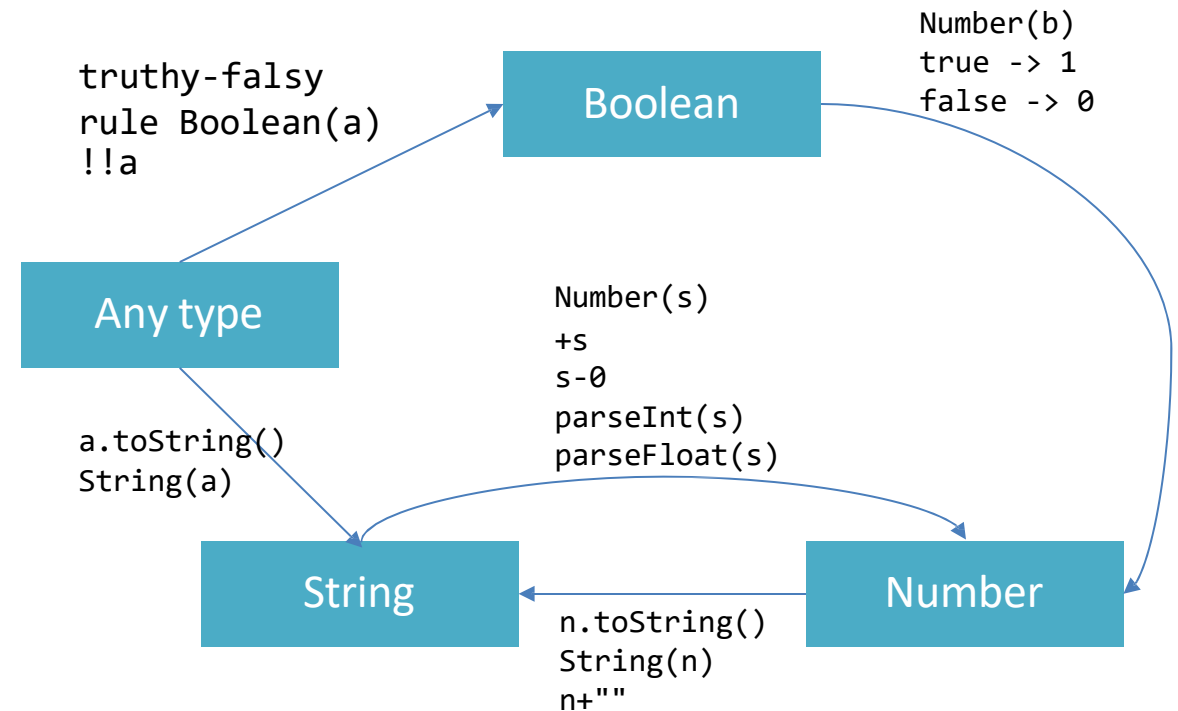
```
> a==b  
false
```

```
const num = 0;  
const obj = new String("0");  
const str = "0";  
  
console.log(num === num); // true  
console.log(num === obj); // false  
console.log(num == obj); // true
```




Automatic Type Conversions

- JS tries to apply type conversions between primitive types, before applying operators
- Some language constructs may be used to “force” the desired conversions
- Using `==` applies conversions
- Using `===` prevents conversions





Logical Operators

Operator	Usage	Description
Logical AND (<code>&&</code>)	<code>expr1 && expr2</code>	Returns <code>expr1</code> if it can be converted to <code>false</code> ; otherwise, returns <code>expr2</code> . Thus, when used with Boolean values, <code>&&</code> returns <code>true</code> if both operands are true; otherwise, returns <code>false</code> .
Logical OR (<code> </code>)	<code>expr1 expr2</code>	Returns <code>expr1</code> if it can be converted to <code>true</code> ; otherwise, returns <code>expr2</code> . Thus, when used with Boolean values, <code> </code> returns <code>true</code> if either operand is true; if both are false, returns <code>false</code> .
Logical NOT (<code>!</code>)	<code>!expr</code>	Returns <code>false</code> if its single operand that can be converted to <code>true</code> ; otherwise, returns <code>true</code> .



Common Operators

- Nullish coalescing (??) operator returns its right-hand side operand when its left-hand side operand is `null` or `undefined`

```
1. const baz = 0 ?? 42;
2. console.log(baz);
3. // Expected output: 0
4. const o5 = "Cat" || "Dog"; // Cat
5. const o6 = false || "Cat"; // Cat
6. const o7 = "Cat" || false; // Cat
7. console.log(typeof 42);
8. // Expected output: "number"
```

- `a || b` : if `a` then `a` else `b`
 - `a`, with default `b`

Addition (+)	Logical AND (&&)
Decrement (--)	Logical OR ()
Division (/)	Logical NOT (!)
Exponentiation (**)	Nullish coalescing operator (??)
Increment (++)	Conditional operator (c ? t : f)
Multiplication (*)	typeof
Remainder (%)	
Subtraction (-)	
Unary negation (-)	
Unary plus (+)	



Function

- A function definition expression typically consists of the keyword `function`
 - Followed by a comma-separated list of zero or more identifiers (the parameter names) in parentheses and
 - Block of JavaScript code (the function body) in curly braces

- ```
let square = function(x) { return x * x; };
```



# Mathematical Functions

## ○ Constants

- Math.E, Math.LN10, Math.LN2, Math.LOG10E, Math.LOG2E, Math.PI, Math.SQRT1\_2, Math.SQRT2

## ○ Functions

- Math.abs(), Math.acos(), Math.acosh(), Math.asin(), Math.asinh(), Math.atan(), Math.atan2(), Math.atanh(), Math.cbrt(), Math.ceil(), Math.clz32(), Math.cos(), Math.cosh(), Math.exp(), Math.expm1(), Math.floor(), Math.fround(), Math.hypot(), Math.imul(), Math.log(), Math.log10(), Math.log1p(), Math.log2(), Math.max(), Math.min(), Math.pow(), Math.random(), Math.round(), Math.sign(), Math.sin(), Math.sinh(), Math.sqrt(), Math.tan(), Math.tanh(), Math.trunc()



JavaScript: The Definitive Guide, 7th Edition  
Chapter 4. Statements

Mozilla Developer Network  
JavaScript Guide » Control Flow and Error Handling  
JavaScript Guide » Loops and Iteration

JavaScript – The language of the Web

# CONTROL STRUCTURES



# Conditional Statements

```
if (condition) {
 statement_1;
} else {
 statement_2;
}
```

if truthy (beware!)

```
if (condition_1) {
 statement_1;
} else if (condition_2) {
 statement_2;
} else if (condition_n) {
 statement_n;
} else {
 statement_last;
}
```

```
switch (expression) {
 case label_1:
 statements_1
 [break;]
 case label_2:
 statements_2
 [break;]
 ...
 default:
 statements_def
 [break;]
}
```

May also be a string



# Loop Statements

```
for ([initialExpression]; [condition]; [incrementExpression]) {
 statement ;
}
```

Usually declares loop  
variable

```
do {
 statement ;
} while (condition);
```

May use break; or  
continue;

```
while (condition) {
 statement ;
}
```





# Special 'for' Statements

```
for (variable in object) {
 statement ;
}
```

- Iterates the variable over all the enumerable **properties** of an object
- Do not use to traverse an array (use numerical indexes, or for-of)

```
for (variable of iterable) {
 statement ;
}
```

- Iterates the variable over all **values** of an iterable object (including Array, Map, Set, string, arguments ...)
- Returns the values, not the keys

```
for(let a in {x: 0, y:3}) {
 console.log(a) ;
}
```

x  
y

```
for(let a of [4,7]) {
 console.log(a) ;
}
```

4  
7

```
for(let a of "hi") {
 console.log(a) ;
}
```

h  
i



# Other Iteration Methods

- Functional programming (strongly supported by JS) allows other methods to iterate over a collection (or any **iterable** object)
  - `a.forEach()`
  - `a.map()`



# Exception Handling

```
try {
 statements ;
} catch(e) {
 statements ;
}
```

```
throw object ;
```

Exception object

```
try {
 statements ;
} catch(e) {
 statements ;
} finally {
 statements ;
}
```

Executed in any case, at  
the end of try and catch  
blocks

EvalError  
RangeError  
ReferenceError  
SyntaxError  
TypeError  
URIError  
DOMException

Contain fields: name, message



JavaScript: The Definitive Guide, 7th Edition  
Chapter 6. Arrays

Mozilla Developer Network  
JavaScript Guide » Indexed Collections

JavaScript – The language of the Web

# ARRAYS



# Creating Arrays

- Array literals
  - `let empty = [];`
- The `...` spread operator on an iterable object
  - `let a = [1, 2, 3];`
  - `let b = [0, ...a, 4];`
- The `Array()` constructor
  - `let a = new Array(10);`
- The `Array.of()` and `Array.from()` factory methods
  - `Array.of(1,2,3) // => [1, 2, 3]`
  - `let copy = Array.from(original);`





# Arrays

- Rich of functionalities
- Elements do not need to be of the same type
- Simplest syntax: `[ ]`
- Property `.length`
- Distinguish between methods that:
  - Modify the array (in-place)
  - Return a new array



# Creating an Array

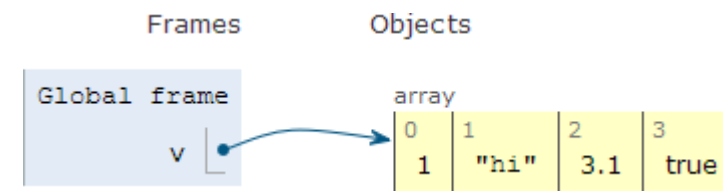
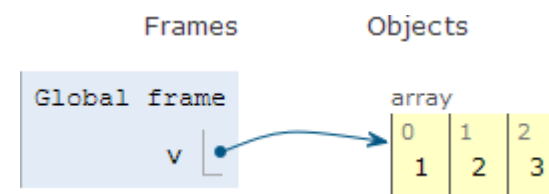
```
let v = [] ;
```

```
let v = [1, 2, 3] ;
```

```
let v = [1, "hi", 3.1, true];
```

```
let v = Array.of(1, 2, 3) ;
```

```
let v = Array.of(1, "hi", 3.1, true) ;
```



- Elements are indexed at positions 0...length-1
- Do not access elements outside range



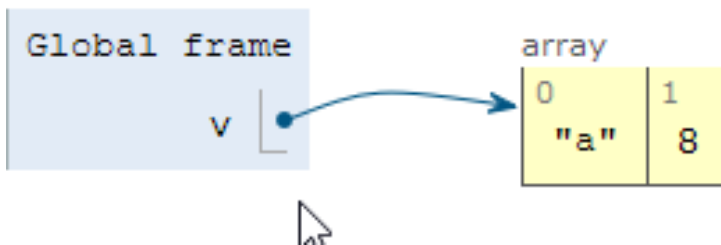
# Adding Elements

- `.length` adjusts automatically

```
let v = [] ;
v[0] = "a" ;
v[1] = 8 ;
v.length // 2
```

Frames

Objects



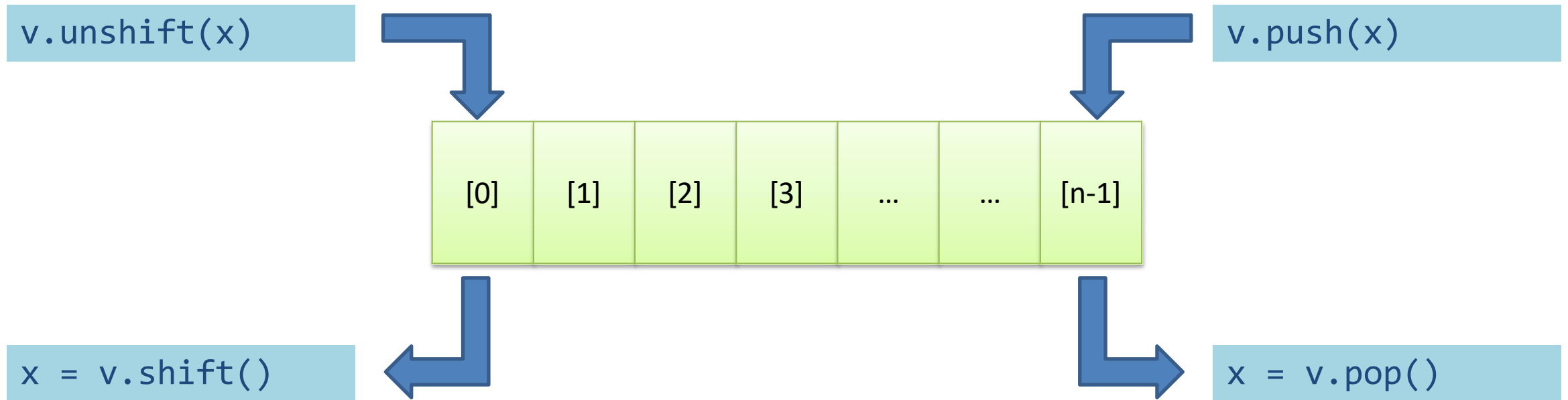
- `.push()` adds at the end of the array
- `.unshift()` adds at the beginning of the array

```
let v = [] ;
v.push("a") ;
v.push(8) ;
v.length // 2
```



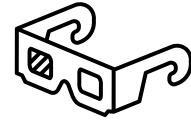
# Adding and Removing from Arrays

○ in-place





# Copying Arrays



```
let v = [] ;
v[0] = "a" ;
v[1] = 8 ;

let alias = v ;
alias[1] = 5 ;
```

```
> console.log(v);
['a', 5]
undefined
> console.log(alias);
['a', 5]
undefined
```

<https://pythontutor.com/javascript.html#mode=edit>

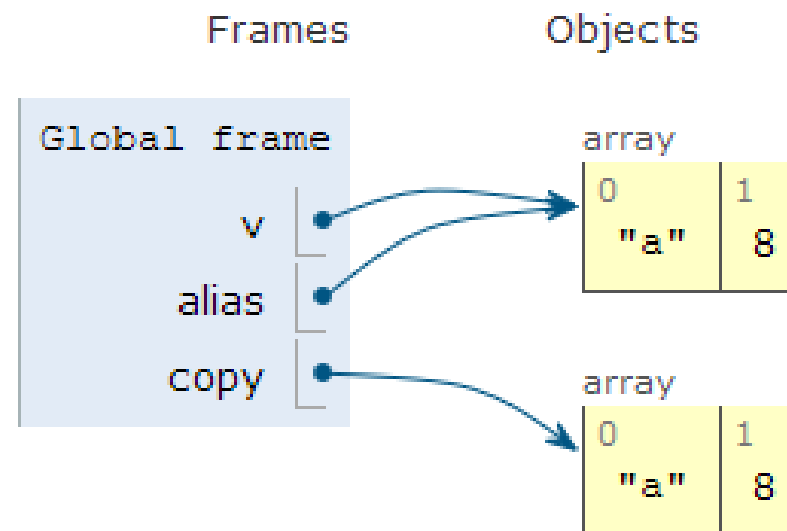


# Copying Arrays

```
let v = [] ;
v[0] = "a" ;
v[1] = 8 ;

let alias = v ;
let copy = Array.from(v) ;
```

- `Array.from` creates a **shallow** copy
- Creates an array from any iterable object





# Iterating over Arrays

Preferred

- Iterators: `for ... of`, `for (... ; ... ; ...)`
- Iterators: `forEach(f)`
  - `f` is a function that processes the element
- Iterators: `every(f)`, `some(f)`
  - `f` is a function that returns true or false: such as `isEven`

```
1. let letters = [..."Hello world"];
2. let everyother = "";
3. for(let [index, letter] of letters.entries()) {
4. if (index % 2 === 0)
5. everyother += letter;
6. // letters at even indexes
7. }
8. everyother // => "Hlowrd"
9.
10. let uppercase = "";
11. letters.forEach(letter => {
12. // Note arrow function syntax here
13. uppercase += letter.toUpperCase(); });
14. uppercase // => "HELLO WORLD"
15.
16. let a = [1,2,3,4,5];
17. a.every(x => x < 10) // => true:
```



# Iterating over Arrays

- Iterators that return a new array: **map(f)**, **filter(f)**
  - **f** works on the element of the array passed as parameter
  - **filter()** method returns an array containing a subset of the elements
- **reduce()**: exec a callback function on all items to progressively compute a result
  - 1<sup>st</sup> argument is the function to perform the reduction operation
  - 2nd argument is an initial value

```
1. let a = [1, 2, 3];
2. a.map(x => x*x) // => [1, 4, 9];
3.
4. let a = [5, 4, 3, 2, 1];
5. a.filter(x => x < 3) // => [2, 1];
6. // values less than 3
7. a.filter((x,i) => i%2 === 0) // => [5, 3, 1];
8. // every other value
9.
10. let a = [1,2,3,4,5];
11. a.reduce((x,y) => x+y, 0) // => 15;
12. a.reduce((x,y) => (x > y) ? x : y) // => 5;
```



# Main Array Methods

```
1. const months = ['Jan', 'March', 'April', 'June'];
2. months.splice(4, 1, 'May');
3. // Replaces 1 element at index 4
4. console.log(months);
5. // Expected output:
6. // Array ["Jan", "Feb", "March", "April", "May"]
```

- **.concat()**
  - joins two or more arrays and returns a new array
- **.join(delimiter = ',')**
  - joins all elements of an array into a (new) string separated by delimiter
- **.slice(start\_index, upto\_index)**
  - extracts a section of an array and returns a new array.
- **.splice(index, count\_to\_remove, addElement1, addElement2, ...)**
  - removes elements from an array and (optionally) replaces them, in place
- **.reverse()**
  - transposes the elements of an array, in place
- **.sort()**
  - sorts the elements of an array in place
- **.indexOf(searchElement[, fromIndex])**
  - searches the array for searchElement and returns the index of the first match
- **.lastIndexOf(searchElement[, fromIndex])**
  - like indexOf, but starts at the end
- **.includes(valueToFind[, fromIndex])**
  - search for a certain value among its entries, returning true or false



# Destructuring Assignment

- Value of the right-hand side of equal signal are extracted and stored in the variables on the left
  - `let [x,y] = [1,2];`
  - `[x,y] = [y,x]; // swap [2.1]`
  - `var foo = ['one', 'two', 'three']; var [one, two, three] = foo;`
- Useful especially with passing and returning values from functions
  - `let [x,y] = toCartesian(r,theta);`



# Spread Operator (3 dots ...)

- Expands an iterable object in its parts, when the syntax requires a comma-separated list of elements
  - `let [x, ...y] = [1,2,3,4]; // we obtain y == [2,3,4]`
  - `const parts = ['shoulders', 'knees'];`
  - `const lyrics = ['head', ...parts, 'and', 'toes'];`
    - `// ["head", "shoulders", "knees", "and", "toes"]`
- Works on the left- and right-hand side of the assignment





# Array to String Conversion

- `join()` method converts all the elements of an array to strings and concatenates them
  - Inverse of the `String.split()` method, which creates an array by breaking a string into pieces
- Arrays, like all JavaScript objects, have a `toString()` method
- Object serialization converts an object's state to a string from which it can later be restored
  - `JSON.stringify()` and `JSON.parse()` serialize and restore JavaScript objects

```
1. let a = [1, 2, 3];
2. a.join() // => "1,2,3"
3. a.join(" ") // => "1 2 3"
4. a.join("") // => "123"

5. [1,2,3].toString() // => "1,2,3"
6. ["a", "b", "c"].toString() // => "a,b,c"
```



JavaScript: The Definitive Guide, 7th Edition  
Chapter 2. Types, Values, and Variables

Mozilla Developer Network  
JavaScript Guide » Text Formatting

JavaScript – The language of the Web

# STRINGS



# Strings in JS

- A string is an immutable ordered sequence of Unicode(\*) characters
- The length of a string is the number of characters it contains (**NOT** bytes)
- JavaScript's strings use zero-based indexing
  - The empty string is the string of length 0
- JavaScript does not have a special type that represents a single character (use length-1 strings)
- String literals may be defined with `'abc'` or `"abc"`
  - Note: when dealing with JSON parsing, only `" "` can be correctly parsed



# String Operations

- All operations always return new strings
  - Consequence of immutability
- `s[3]`: indexing
- `s1 + s2`: concatenation
- `s.length`: number of characters
  - Note: `.length` , not `.length()`



# String Methods

| Method                                                                                     | Description                                                                                                                     |
|--------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| <code>charAt</code> , <code>charCodeAt</code> ,<br><code>codePointAt</code>                | Return the character or character code at the specified position in string.                                                     |
| <code>indexOf</code> , <code>lastIndexOf</code>                                            | Return the position of specified substring in the string or last position of specified substring, respectively.                 |
| <code>startsWith</code> , <code>endsWith</code> ,<br><code>includes</code>                 | Returns whether or not the string starts, ends or contains a specified string.                                                  |
| <code>concat</code>                                                                        | Combines the text of two strings and returns a new string.                                                                      |
| <code>fromCharCode</code> , <code>fromCodePoint</code>                                     | Constructs a string from the specified sequence of Unicode values. This is a method of the String class, not a String instance. |
| <code>split</code>                                                                         | Splits a <code>String</code> object into an array of strings by separating the string into substrings.                          |
| <code>slice</code>                                                                         | Extracts a section of a string and returns a new string.                                                                        |
| <code>substring</code> , <code>substr</code>                                               | Return the specified subset of the string, either by specifying the start and end indexes or the start index and a length.      |
| <code>match</code> , <code>matchAll</code> , <code>replace</code> ,<br><code>search</code> | Work with regular expressions.                                                                                                  |
| <code>toLowerCase</code> , <code>toUpperCase</code>                                        | Return the string in all lowercase or all uppercase, respectively.                                                              |
| <code>normalize</code>                                                                     | Returns the Unicode Normalization Form of the calling string value.                                                             |
| <code>repeat</code>                                                                        | Returns a string consisting of the elements of the object repeated the given times.                                             |
| <code>trim</code>                                                                          | Trims whitespace from the beginning and end of the string.                                                                      |



# Unicode Issues

- Strings are a sequence of 16-bit Unicode 'code units'
  - Fine for all Unicode characters from `0000` to `FFFF`
  - Characters ('graphemes') from `010000` to `10FFFF` are represented by a pair of code units (and they occupy 2 index positions)
  - Therefore, not all string methods work well with Unicode characters above `FFFF` (e.g., emojis, flags, ...)

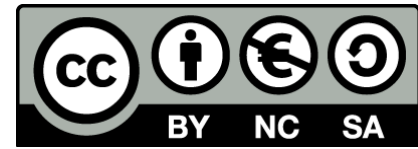


# Template Literals

- Strings included in `backticks` can embed expressions delimited by `\${}`
- The value of the expression is interpolated into the string
  - `let name = "Bill";`
  - `let greeting = `Hello ${ name }.`; // greeting == "Hello Bill."`
- Very useful and quick for string formatting
- Template literals may also span multiple lines



# License



- These slides are distributed under a Creative Commons license “**Attribution-NonCommercial-ShareAlike 4.0 International (CC BY-NC-SA 4.0)**”
- **You are free to:**
  - **Share** — copy and redistribute the material in any medium or format
  - **Adapt** — remix, transform, and build upon the material
  - The licensor cannot revoke these freedoms as long as you follow the license terms.
- **Under the following terms:**
  - **Attribution** — You must give [appropriate credit](#), provide a link to the license, and [indicate if changes were made](#). You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.
  - **NonCommercial** — You may not use the material for [commercial purposes](#).
  - **ShareAlike** — If you remix, transform, or build upon the material, you must distribute your contributions under the [same license](#) as the original.
  - **No additional restrictions** — You may not apply legal terms or [technological measures](#) that legally restrict others from doing anything the license permits.
- <https://creativecommons.org/licenses/by-nc-sa/4.0/>

